

Autenticação com JWT

DIM0547 — Sprint 3

Prof. Fernando · UFRN · 2026.1

De onde paramos

Sprint 2: a API **persiste** dados de verdade — PostgreSQL, sqlc, JOINS 1:N.

Já temos	Falta
CRUD persistente	Saber quem está chamando
Relacionamentos 1:N	Proteger rotas que não são públicas
Endpoints REST idiomáticos	Distinguir "qualquer um" de "o dono do recurso"

Hoje a API ganha **identidade**: login que devolve um token, e rotas que só respondem para quem apresenta um token válido.

Plano da aula

1. Autenticação vs autorização — não confundir ~5 min
2. O problema: HTTP é sem estado ~10 min
3. Anatomia de um JWT: três partes ~15 min
4. Assinatura: por que ninguém forja o token ~10 min
5. Fluxo de login + middleware de auth em Go ~20 min
6. Erros comuns e o que cai na Sprint 3 ~10 min

Autenticação ≠ autorização

	Pergunta	Exemplo
Autenticação	Quem é você?	"Sou a Maria, aqui está minha senha"
Autorização	Você pode fazer isso?	"Maria pode editar o contato 7?"

A ordem importa: primeiro **autentica** (estabelece identidade), depois **autoriza** (verifica permissão).

JWT resolve principalmente a **autenticação** — carrega a identidade de forma confiável. A autorização é o que **você** decide fazer com a identidade que o token carrega (papel, dono do recurso, escopo).

0 problema: HTTP não tem memória

Cada requisição HTTP é **independente**. O servidor não "lembra" que você fez login na requisição anterior.

```
POST /login      → "ok, você é a Maria"   (servidor esquece)
GET  /contacts  → "...e você é quem?"         (de novo do zero)
```

Duas famílias de solução:

Abordagem	Onde fica o estado
Sessão no servidor	Servidor guarda; cliente só tem um <code>session_id</code>
Token autocontido (JWT)	O token carrega a identidade; servidor não guarda nada

JWT é **stateless**: o servidor não precisa de uma tabela de sessões. Isso escala horizontalmente sem esforço — qualquer instância valida o token sozinha.

Sessão vs JWT: o trade-off

Sessão no servidor

- Estado no servidor (memória ou Redis)
- Revogar é trivial: apaga a sessão
- Precisa de armazenamento compartilhado entre instâncias
- Cliente carrega só um ID opaco

JWT (stateless)

- Estado no **token**, no cliente
- Escala sem armazenamento compartilhado
- Revogar é **difícil** — o token é válido até expirar
- Cliente carrega a identidade inteira

Não existe escolha "certa" universal. JWT é bom em APIs distribuídas; o controle de sessão é uma boa quando revogação imediata é requisito. Na Sprint 3 usamos JWT — e na próxima aula vemos como **refresh tokens** atenuam o problema da revogação.

JWT: o que é, concretamente

Um **JSON Web Token** é uma string de três partes separadas por ponto:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxIiwibmFtZSI6Ikk1hcmIhIn0.dQ7...  
└── header ─┘ └────────── payload ─────────┘ └── signature ─┘
```

HEADER.PAYLOAD.SIGNATURE

Parte	Conteúdo	Codificação
Header	algoritmo e tipo	Base64URL de um JSON
Payload	as <i>claims</i> (a identidade)	Base64URL de um JSON
Signature	a prova de autenticidade	bytes assinados, Base64URL

Base64URL **não é criptografia**. Qualquer pessoa decodifica o payload e lê o conteúdo. Quem garante a integridade é a **assinatura** — não o sigilo.

Header e Payload

```
// HEADER  
{ "alg": "HS256", "typ": "JWT" }
```

```
// PAYLOAD – as claims  
{  
  "sub": "1",                // subject: ID do usuário  
  "name": "Maria",  
  "role": "user",  
  "iat": 1716120000,         // issued at  
  "exp": 1716123600         // expiration  
}
```

Nunca coloque dado sensível no payload (senha, número de cartão). O payload é **legível por qualquer um**. Coloque apenas o necessário para identificar e autorizar: ID, papel, talvez escopo.

Claims registradas (RFC 7519)

Algumas claims têm significado padronizado — bibliotecas as validam automaticamente:

Claim	Significado	Por que importa
sub	Subject — quem o token representa	É o "quem é você"
iss	Issuer — quem emitiu	Validar a origem do token
exp	Expiration — quando expira	Token velho é rejeitado
iat	Issued at — quando foi emitido	Idade do token
nbf	Not before — válido a partir de	Token "agendado"

`exp` é a claim mais importante para a segurança prática. Sem ela, o token vale **para sempre** — um vazamento vira um problema permanente. Tokens de acesso devem ser **curtos** (minutos).

A assinatura: o coração do JWT

```
signature = HMAC-SHA256(  
    base64url(header) + "." + base64url(payload),  
    SECRET  
)
```

A assinatura é função de **três coisas**: o header, o payload e um **segredo** que só o servidor conhece.

Alguém tenta...	Resultado
Mudar <code>"role": "user"</code> para <code>"admin"</code>	A assinatura não confere mais
Recalcular a assinatura	Não tem o <code>SECRET</code> — impossível
Reusar a assinatura antiga com payload novo	Assinatura é de outro payload — não confere

Qualquer alteração de 1 byte no header ou payload **invalida** a assinatura. É isso que torna o JWT confiável mesmo sendo legível: você pode **ler**, mas não pode **forjar**.

HS256 vs RS256

	HS256 (simétrico)	RS256 (assimétrico)
Chaves	Um segredo compartilhado	Par: privada assina, pública verifica
Quem pode emitir	Quem tem o segredo	Só quem tem a chave privada
Quem pode verificar	Quem tem o segredo	Qualquer um com a chave pública
Quando usar	API monolítica, um emissor	Vários serviços verificam, um emite

```
// HS256 – o que usamos na Sprint 3  
token.SignedString([]byte(secret))
```

Para o projeto, **HS256 basta**: vocês são emissor e verificador ao mesmo tempo. RS256 entra quando o emissor (ex.: um Identity Provider) é separado de quem consome o token.

0 ataque `alg: none`

Um JWT mal-validado aceita um header assim:

```
{ "alg": "none", "typ": "JWT" }
```

`alg: none` significa "token sem assinatura". Uma biblioteca ingênua **aceita** — e aí qualquer um forja qualquer payload.

```
// ✅ sempre declare o método esperado e cheque-o
token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (any, error) {
    if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
        return nil, fmt.Errorf("alg inesperado: %v", t.Header["alg"])
    }
    return []byte(secret), nil
})
```

Regra de ouro: **o servidor decide qual algoritmo aceita**, nunca o token. Confiar no `alg` do header é confiar no atacante.

0 fluxo completo

1. POST /login {email, senha}
- ↓
2. Servidor: confere senha (`bcrypt.CompareHashAndPassword`)
- ↓
3. Servidor: gera JWT assinado → devolve no corpo da resposta
- ↓
4. Cliente guarda o token e o envia em CADA requisição:
Authorization: Bearer eyJhbGc...
- ↓
5. Middleware: valida assinatura + exp → extrai claims
- ↓
6. Handler: já sabe quem é o usuário (via context)

Passo 2-3: o handler de login

```
func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
    var req LoginRequest
    json.NewDecoder(r.Body).Decode(&req)

    user, err := h.repo.FindByEmail(r.Context(), req.Email)
    if err != nil {
        http.Error(w, "credenciais inválidas", http.StatusUnauthorized)
        return
    }
    // senha guardada como HASH – nunca em texto puro
    err = bcrypt.CompareHashAndPassword(user.PasswordHash, []byte(req.Password))
    if err != nil {
        http.Error(w, "credenciais inválidas", http.StatusUnauthorized)
        return
    }
    token, _ := generateToken(user)
    writeJSON(w, map[string]string{"access_token": token})
}
```

Mensagem de erro **genérica** ("credenciais inválidas") tanto para email inexistente quanto para senha errada. Diferenciar as duas vaza informação: revela quais emails existem.

Gerando o token

```
func generateToken(user User) (string, error) {
    claims := jwt.MapClaims{
        "sub":  fmt.Sprint(user.ID),
        "name": user.Name,
        "role": user.Role,
        "iat":  time.Now().Unix(),
        "exp":  time.Now().Add(15 * time.Minute).Unix(),
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(jwtSecret))
}
```

Pontos a observar:

- `exp` com **15 minutos** — token de acesso é curto por design
- `jwtSecret` vem de **variável de ambiente**, nunca hard-coded no repositório
- `role` na claim — é o que o middleware de autorização vai ler depois

Passo 5: o middleware de autenticação

```
func AuthMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        header := r.Header.Get("Authorization")  
        tokenStr, ok := strings.CutPrefix(header, "Bearer ")  
        if !ok {  
            http.Error(w, "token ausente", http.StatusUnauthorized)  
            return  
        }  
        claims, err := validateToken(tokenStr)  
        if err != nil {  
            http.Error(w, "token inválido", http.StatusUnauthorized)  
            return  
        }  
        ctx := context.WithValue(r.Context(), userKey, claims)  
        next.ServeHTTP(w, r.WithContext(ctx)) // ← segue o fluxo  
    })  
}
```

O middleware é o **portão**. Se o token não é válido, a requisição **morre aqui** — o handler protegido nunca executa. Repare no padrão `func(http.Handler) http.Handler` da Sprint 0.

Validando o token

```
func validateToken(tokenStr string) (jwt.MapClaims, error) {
    token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (any, error) {
        if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, fmt.Errorf("alg inesperado")
        }
        return []byte(jwtSecret), nil
    })
    if err != nil || !token.Valid {
        return nil, fmt.Errorf("token inválido")
    }
    return token.Claims.(jwt.MapClaims), nil
}
```

A chamada `jwt.Parse` valida, em uma só passada: a **assinatura**, o **exp** (token expirado → erro) e o **alg** (via callback). Se qualquer uma falha, `token.Valid` é `false`.

Aplicando o middleware com Chi

```
r := chi.NewRouter()

// rotas públicas
r.Post("/login", authHandler.Login)
r.Post("/register", authHandler.Register)

// rotas protegidas – grupo com o middleware
r.Group(func(r chi.Router) {
    r.Use(AuthMiddleware)           // ← portão para tudo abaixo
    r.Get("/contacts", listContacts)
    r.Post("/contacts", createContact)
    r.Delete("/contacts/{id}", deleteContact)
})
```

Mesma técnica de `r.Group` da Sprint 1: um middleware aplicado a um **subconjunto** de rotas. Login e register **ficam de fora** — senão ninguém conseguiria autenticar para entrar.

Lendo a identidade no handler

```
func listContacts(w http.ResponseWriter, r *http.Request) {  
    claims := r.Context().Value(userKey).(jwt.MapClaims)  
    userID := claims["sub"].(string)  
    role    := claims["role"].(string)  
  
    // agora o handler decide:  
    // - filtrar contatos só do userID?  
    // - exigir role == "admin"?  
    ...  
}
```

O middleware fez	O handler faz
Provou quem é (autenticação)	Decide o que pode (autorização)
Pôs as claims no <code>context</code>	Lê as claims e aplica a regra de negócio

Separação limpa: o middleware não conhece regra de negócio; o handler não reimplementa validação de token. Cada um faz uma coisa.

Armadilha: o nome do parâmetro errado

`r.Context().Value(userKey)` só funciona se a **chave** for a mesma na escrita e na leitura.

```
// ✗ string crua como chave – colide com outros pacotes
ctx := context.WithValue(r.Context(), "user", claims)

// ✔ tipo próprio, não exportado – chave única e segura
type contextKey string
const userKey contextKey = "user"
ctx := context.WithValue(r.Context(), userKey, claims)
```

`context.WithValue` com chave `string` é um *code smell* que o `go vet` reclama. Use um tipo próprio: garante que nenhum outro pacote sobrescreva sua chave por acidente.

Erros comuns

Sintoma	Causa provável
Token "válido" mas servidor rejeita	Segredo diferente entre gerar e validar
Funciona local, falha no CI	<code>JWT_SECRET</code> não definido como env var no workflow
Qualquer token passa	Esqueceu de checar <code>token.Valid</code> ou o <code>alg</code>
<code>401</code> em rota que deveria ser pública	Middleware aplicado no grupo errado
Token nunca expira	Faltou a claim <code>exp</code> no <code>MapClaims</code>
<code>panic</code> ao ler claim	Type assertion sem o <code>, ok</code> — claim ausente

Senhas: bcrypt, nunca texto puro

```
// no register – guardar
hash, _ := bcrypt.GenerateFromPassword([]byte(senha), bcrypt.DefaultCost)
// guarda `hash` no banco, descarta a senha

// no login – conferir
err := bcrypt.CompareHashAndPassword(user.PasswordHash, []byte(senha))
```

- `bcrypt` é **lento de propósito** — dificulta ataques de força bruta
- Inclui um *salt* automático — duas senhas iguais geram hashes diferentes
- A senha em texto puro **nunca** toca o banco, nem o log

Vazamento de banco acontece. Se as senhas estiverem em texto puro, é catástrofe. Em hash bcrypt, o atacante ainda tem muito trabalho pela frente. Isso conecta com o **OWASP** que veremos na aula 11.

Checklist da aula

A Sprint 3 entrega os seguintes itens (prazo sprint: 03/06) :

- [] `POST /login` confere senha com `bcrypt` e devolve JWT
- [] Token assinado com `HS256` e segredo em **env var**
- [] Claim `exp` presente — token de acesso curto
- [] `AuthMiddleware` valida assinatura, `exp` e `alg`
- [] Rotas sensíveis dentro de `r.Group` com o middleware
- [] Handler lê a identidade via `context`

Referências

JWT

- [RFC 7519 — JSON Web Token](#) — a especificação
- [jwt.io](#) — decodificar e inspecionar tokens
- `golang-jwt/jwt` — a biblioteca usada no projeto

Segurança

- `golang.org/x/crypto/bcrypt` — hashing de senhas
- [OWASP](#) — Livro sobre segurança em Go - boas práticas